Python Programming for Life Science Students (PyLifeS)

Raghunathan Ramakrishnan
Tata Institute of Fundamental Research Hyderabad
Hyderabad, INDIA
ramakrishnan@tifrh.res.in

Content

Topic		Page
1. Python syntax and data types	…	5
2. Numerical arrays with numpy	…	12
3. Visualization with matplotlib	…	16
4. Elementary data analysis with Pandas	…	20
5. Mathematical modeling	…	35

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

References

- *General References*
  - K. Behrman, *Foundational Python for Data Science*, Pearson Education, 2023
  - A. Downey, *Think Python*, O'reilly, 2022
  - A. Hofmann, S. Clokie, *Wilson and Walker's Principles and Techniques of Biochemistry and Molecular Biology*, Cambridge University Press, 2018
  - E. N. Bodine, S. Lenhart, L. J. Gross, *Mathematics for the Life Sciences*, Princeton University Press, 2014

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

1. Python syntax and data types

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Working with Jupyter notebook

- *Launching Jupyter via Anaconda*
  - Open **Anaconda Navigator**
  - Click **Launch** under **Jupyter Notebook**
  - A browser window will open showing your file system.
  - Navigate to your working directory. On the right side of the screen, click the **`New`** button and select **`Python 3 (ipykernel)`** from the dropdown menu. This opens a new notebook.

- *Installing and launching Jupyter with pip*
  - Open a terminal and run: **`pip install notebook`**
  - Once installed, launch Jupyter with: **`jupyter notebook`**

- *Jupyter notebook opens in a web browser*
  - When you launch Jupyter Notebook, it opens in your *default web browser* (e.g., Chrome, Firefox, Safari) as a tab or window.

- *Anatomy of a notebook*
  - A Jupyter Notebook is organized into **cells**:
  - **Code cells**: Where you write and run Python code
  - **Markdown cells:** Where you write explanations, equations, and documentation

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Getting started with Python

- *Assignment and print statements*

```
x = 22/7                                    # This is an assignment statement

print(x)                                    # Print statement
```

```
3.142857142857143
```

```
print("The value of x is:", x)              # Print statement with some text
print("The value of x is:", round(x, 7))    # Round-off x to 7 decimal places for printing
print(f"The value of x is: {x:15.7f}")      # Print x in 15 width and use 7 spaces for decimal part
```

```
The value of x is: 3.142857142857143
The value of x is: 3.1428571
The value of x is:       3.1428571
```

```
x = 22/7
y = 7/22

print(x, y)
```

```
3.142857142857143 0.3181818181818182
```

- In this presentation, multiple statements may appear on a single line, separated by a semicolon (;).
- When working in a Jupyter Notebook, you should write each statement on a separate line for better readability.

```
x = 22/7; y = 7/22                          # Two statements separated by semi-colon

print(x, y)
```

```
3.142857142857143 0.3181818181818182
```

```
x, y, z = 22/7, 7/22, 22/7 + 7/22           # Multiple assignments in one line

print(f"x is: {x:.7f}, y is: {y:.7f}, z is: {z:.7f}")
```

```
x is: 3.1428571, y is: 0.3181818, z is: 3.4610390
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Math operations

- *Let's answer these questions using Python*

  - You are preparing a buffer by mixing 50 µL of stock solution with 150 µL of water. What is the total volume of the final solution?

```
# Addition
stock_volume = 50
water_volume = 150
total_volume = stock_volume + water_volume
print(total_volume)
```

```
200
```

  - A culture initially contained 40 cells. After incubation, the culture contains 20,000 cells. What is the net increase in the number of cells?

```
# Subtraction
final_cells = 20000
initial_cells = 40
net_increase = final_cells - initial_cells
print(net_increase)
```

```
19960
```

  - A bacterial culture has a concentration of 2,000 cells per mL. If the total culture volume is 100 mL, how many total cells are present in the culture?

```
# Multiplication
cells_per_ml = 2000
volume_ml = 100
total_cells = cells_per_ml * volume_ml
print(total_cells)
```

```
200000
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- You count 420 colonies on an agar plate. The sample plated corresponds to a 1:100 dilution (dilution factor = 0.01). What is the original cell concentration (cells per mL) of the culture?

```
# Division
colonies = 420
dilution = 0.01
cells_per_ml = colonies / dilution
print(cells_per_ml)
```

```
42000.0
```

- A bacterial culture starts with 2,000 cells. If the population increases ten-fold per generation, how many cells will be present after 3 generations?

```
# Exponentiation
initial_cells = 2000
growth_factor = 10
generations = 3
final_cells = initial_cells * (growth_factor**generations)
print(final_cells)
```

```
2000000
```

- A bacterial culture grows for 10 hours. If the generation time is 3 hours, how many complete generations occur during this period?

```
# Floor division
time_hours = 10
generation_time = 3
generations = time_hours // generation_time
print(generations)
```

```
3
```

- You have 23 samples that need to be placed equally into 5 tubes. After filling as many tubes as possible with equal numbers of samples, how many samples are left over?

```
# Modulus
samples = 23
tubes = 5
leftover = samples % tubes
print(leftover)
```

```
3
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Data types

- *Numeric / Sequence / Mapping / Boolean*
  - Integer, float, complex / string, list, tuple / dictionary / bool (True, False)

```
a = 5
print(a, type(a))

b = 5.0
print(b, type(b))

c = 2 + 4j
print(c, type(c))

d = complex(2,4)
print(d, type(d))

e = 'One is 1'
print(e, type(e))
```

```
5 <class 'int'>
5.0 <class 'float'>
(2+4j) <class 'complex'>
(2+4j) <class 'complex'>
One is 1 <class 'str'>
```

```
x = 8

print(x > 5 and x < 10)   # True
print(x < 5 or x > 10)    # False
print(not (x == 8))       # False

TruthValue = 8 < 7
print(type(TruthValue))
```

```
True
False
False
<class 'bool'>
```

- *Dictionary*
  - To store key/value pairs

```
params={'mu':1.0, 'sigma':0.5}
print(params, type(params))
```

```
{'mu': 1.0, 'sigma': 0.5} <class 'dict'>
```

```
print(len(params))
print(params.keys())
print(params.values())
```

```
2
dict_keys(['mu', 'sigma'])
dict_values([1.0, 0.5])
```

```
params.update({'scal':1.0})
print(params)
```

```
{'mu': 1.0, 'sigma': 0.5, 'scal': 1.0}
```

```
params['scal'] # query by key returns the value
```

```
1.0
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

String manipulation

- *Splicing, splitting*

```
mystr='Life will find its way!'
a=mystr[0]
b=mystr[0:3] # slice from index 0 up to (but not including) index 3
print(a,",",b)
```

```
L , Lif
```

```
mystr='1,2,3,4'
a=mystr.split(',')  # output is a list
b=mystr.split(',')[0:2] # first two entries of the list
print(a,b)
mystr='ATAAATCT'
sub_seqs=mystr.split('T')
print(sub_seqs)
```

```
['1', '2', '3', '4'] ['1', '2']
['A', 'AAA', 'C', '']
```

```
print(len(sub_seqs), type(sub_seqs))
```

```
4 <class 'list'>
```

```
mystr='ATAAATCT'
mystr[2:2]='G' # Strings in Python are immutable
print(mystr)
```

```
---------------------------------------------------
TypeError                                 Traceb
Cell In[94], line 2
      1 mystr='ATAAATCT'
----> 2 mystr[2:2]='G' # Strings in Python are i
      3 print(mystr)

TypeError: 'str' object does not support item as
```

```
sub_seqs[0]+sub_seqs[1]+sub_seqs[2]  # '+'' is for concatenation or combining
```

```
'AAAAC'
```

```
N=len(sub_seqs)
for i in range(0,N):
    print(i,sub_seqs[i])    # NOTE: 4 empty spaces for intendation
```

```
0 A
1 AAA
2 C
3
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

List

```
string='What is Life?'
sub_str=string.split(',') # This is a list with 3 elements

print(sub_str, type(sub_str), len(sub_str))
```

```
['What is Life?'] <class 'list'> 1
```

```
A=[1,2,3]  # list of integers
B=[1,'one','1.0']  # list can contain different datatypes
print(A[0:2]) # last index, excluded
```

```
[1, 2]
```

Escape sequence for special characters

```
# String concatenation
str1 = 'neuron'
str2 = 'doesn\'t' # same as "doesn't"
str3 = 'divide'
print(str1+str2+str3)          # Concatenate with +
print(str1+' '+str2+' '+str3)
```

```
neurondoesn'tdivide
neuron doesn't divide
```

```
# Append list
str4=[str1,str2]
str5=[str3]
str4+str5     # + for appending a list to another
```

```
['neuron', "doesn't", 'divide']
```

```
t = (1, 2, 3) # a tuple is like a list, but immutable
t[1]=4
```

- Because tuples are immutable, they are often used to store function arguments and return multiple values safely.

- *Always deep copy a list*

```
A=[1,2,3,4]

B=A # is a shallow copy
print("List A is:", A, "List B is:", B)

B[2]=7  # change a value in B, which also affects A
print("List A is:", A, "List B is:", B)
```

```
List A is: [1, 2, 3, 4] List B is: [1, 2, 3, 4]
List A is: [1, 2, 7, 4] List B is: [1, 2, 7, 4]
```

```
A=[1,2,3,4]

B=A.copy() # is a deep copy
print("List A is:", A, "List B is:", B)

B[2]=7  # change a value in B, which does not affect A
print("List A is:", A, "List B is:", B)
```

```
List A is: [1, 2, 3, 4] List B is: [1, 2, 3, 4]
List A is: [1, 2, 3, 4] List B is: [1, 2, 7, 4]
```

```
# assigning a slice works differently
A=[1,2,3,4]; B=A[0:3]  # deep copy
B[2]=7; print("List A is:", A, "List B is:", B)

A=[1,2,3,4]; B=A[0:3].copy()
B[2]=7; print("List A is:", A, "List B is:", B)

import numpy as np
A=np.array([1,2,3,4]); B=A[0:3] # shallow copy
B[2]=7; print("Array A is:", A, "Array B is:", B)

A=np.array([1,2,3,4]); B=A[0:3].copy()
B[2]=7; print("Array A is:", A, "Array B is:", B)
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

2. Numerical arrays with numpy

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Numpy

- *Numpy arrays*
  - Useful for most numerical calculations and plotting

```
import numpy as np
```

```
A=[1,2,3]; A=np.array(A) # same as A=np.array([1,2,3]), A is a 1D array
```

```
print(A.shape)
```

```
(3,)
```

```
element_first=A[0]; element_last=A[-1]; print(element_first, element_last)
```

```
2 1
```

```
A=np.array([[1,2,3],[4,5,6]]); print(A); print(A.shape) # A is a 2D array; 2x3 matrix
```

```
[[1 2 3]
 [4 5 6]]
(2, 3)
```

```
row_0=A[0,:]; print(row_0)
```

```
[1 2 3]
```

```
col_1=A[:,1]; print(col_1)
```

```
[2 5]
```

```
Nrow=A.shape[0]; Ncol=A.shape[1]; print(Nrow, Ncol)
```

```
2 3
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- *More on array indices*
  - Try the following cell-by-cell

```
A=np.array([2,3,1,3,1]); print(A)
```

```
print(A[:])      # all elements (full slice; start and stop are default)
```

```
print(A[::+1])   # all elements with explicit step = +1 (same as A[:])
```

```
print(A[::-1])   # all elements in reverse order (step = -1)
```

```
A_sorted_asc=np.sort(A); print(A_sorted_asc)
```

```
A_sorted_desc=A_sorted_asc[::-1]; print(A_sorted_desc) # [::-1] means all row, all col
```

```
index_A_asc=np.argsort(A); print(index_A_asc)
```

```
print('Minimum value of A is:',A_sorted_asc[0])

index_small=index_A_asc[0]
print('Minimum value of A is:',A[index_small],' with index ',index_small)
```

```
print('Maximum value of A is:',A_sorted_asc[-1])
print('Maximum value of A is:',A_sorted_desc[0])

index_big=index_A_asc[-1]
print('Maximum value of A is:',A[index_big],' with index ',index_big)
```

```
index_A_desc=index_A_asc[::-1]; print(index_A_desc)
```

```
index_big=index_A_desc[0]
print('Maximum value of A is:',A[index_big],' with index ',index_big)
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- *Creating (or initializing) numpy arrays*
  - Try the following cell-by-cell

```
x = np.zeros(3); print(x)
```

```
x = np.ones(3); print(x)
```

```
x = np.zeros((4,3)); print(x)
```

```
x = np.zeros(3, dtype='int'); print(x)
```

```
x = np.zeros(3, dtype='float'); print(x)
```

```
# create an array of given size with evenly spaced values
xi, xf, N_x = 0, 1, 10 # initial, final, number
x = np.linspace(xi, xf, N_x); print(x)
```

```
# creates an array with evenly spaced values of given step size
xi, xf, dx = 0, 1, 0.1 # initial, final, step
x = np.arange(xi, xf, dx); print(x)
```

```
# array with uniformly sampled random numbers
x_rand1 = np.random.rand(2); print(x_rand1,'\n','-'*20)
x_rand2 = np.random.rand(2,2); print(x_rand2,'\n','-'*20)
x_rand2 = np.random.rand(2,2,2); print(x_rand1,'\n','-'*20)
```

```
np.random.choice(x_rand1) # choice method gets one random value from a 1D array
```

```
x = np.zeros(3, dtype='int16'); print(x) # 16-bit (2-byte) per element array with zeros
```

```
print(np.iinfo(np.int16).min, np.iinfo(np.int16).max) # -2**15, 2**15-1
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

3. Visualization with matplotlib

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

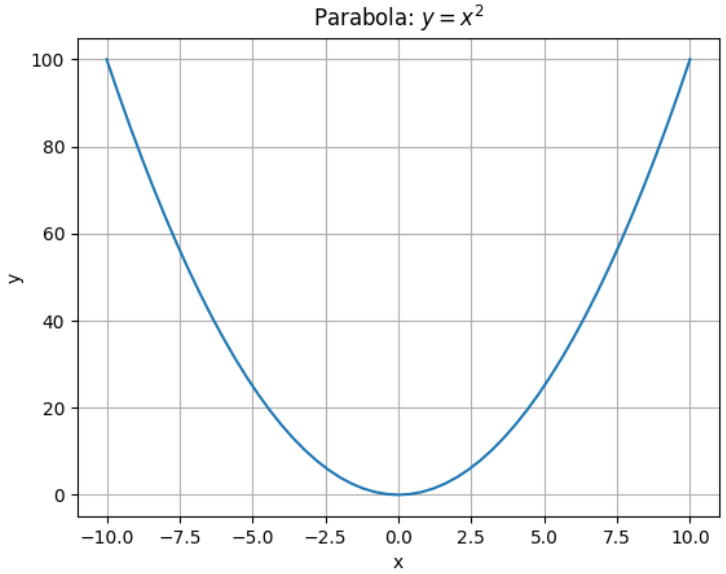
Basic plotting with **matplotlib.pyplot**

```
import numpy as np
import matplotlib.pyplot as plt

xmin = -10; xmax = 10; Ngrids = 51

x = np.linspace(xmin, xmax, Ngrids)
y = x**2

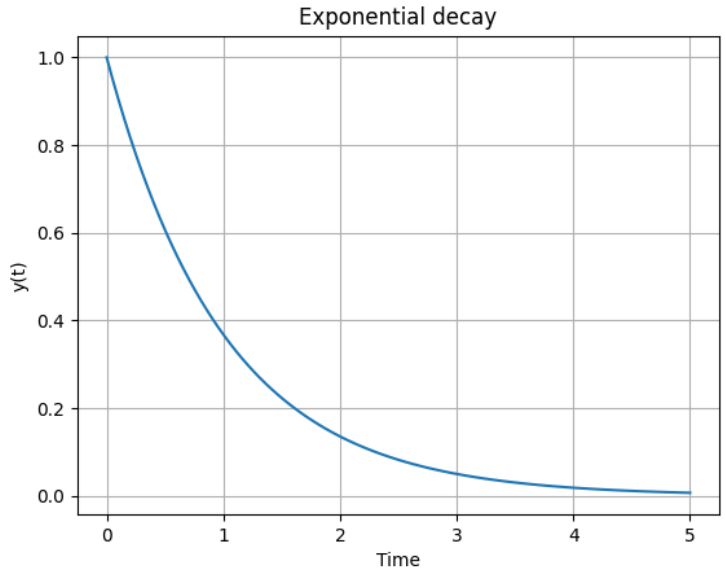
plt.plot(x, y)
plt.title("Parabola: $y = x^2$")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.savefig('parabola.png') # png, pdf, or svg
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 5, 100)
y = np.exp(-t)

plt.plot(t, y)
plt.title("Exponential decay")
plt.xlabel("Time")
plt.ylabel("y(t)")
plt.grid()
plt.show()
```



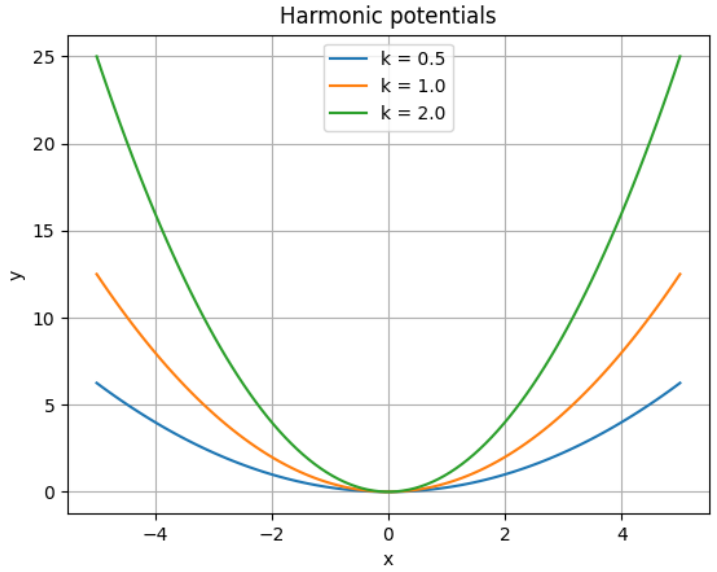
© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)

k=0.5; plt.plot(x, 0.5*k*x**2, label=f"k = {k}")
k=1.0; plt.plot(x, 0.5*k*x**2, label=f"k = {k}")
k=2.0; plt.plot(x, 0.5*k*x**2, label=f"k = {k}")

plt.title("Harmonic potentials")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```

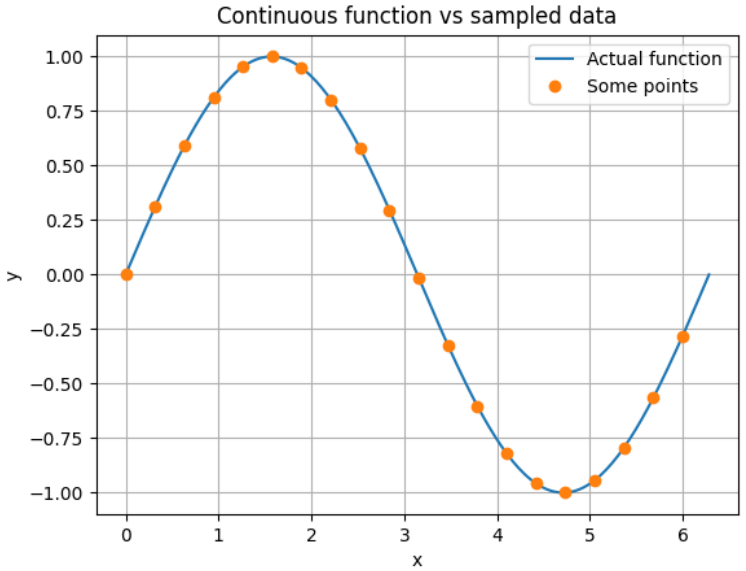



```python
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2*np.pi, 200); y = np.sin(x)
a=x[::10]; b= y[::10] # a=x[start : stop : step], pick every 10th po

plt.plot(x, y, label="Actual function")
plt.plot(a, b, 'o', label="Some points")

plt.title("Continuous function vs sampled data")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

parametric plot

```
import numpy as np
import matplotlib.pyplot as plt

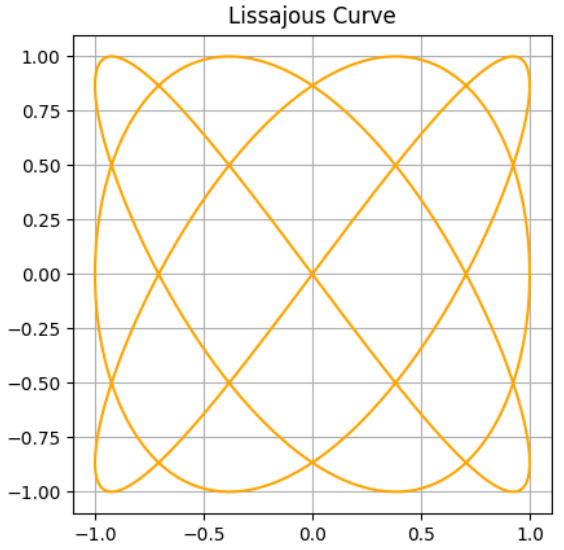
t = np.linspace(-np.pi, np.pi, 500)

A = 1; B = 1; a = 3; b = 4

x = A*np.sin(a*t + np.pi/2)
y = B*np.sin(b*t)

fig, ax = plt.subplots()  # create figure and axes
ax.plot(x, y, color="orange")
ax.set_aspect("equal")   # equal scaling: 1 unit in x = 1 unit in y

plt.title("Lissajous Curve")
plt.grid()
plt.show()
```



contour plot

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

def f(x, y):
    return x**2 + y**2

x = np.arange(-2.0, 2.0, 0.1)
y = np.arange(-2.0, 2.0, 0.1)

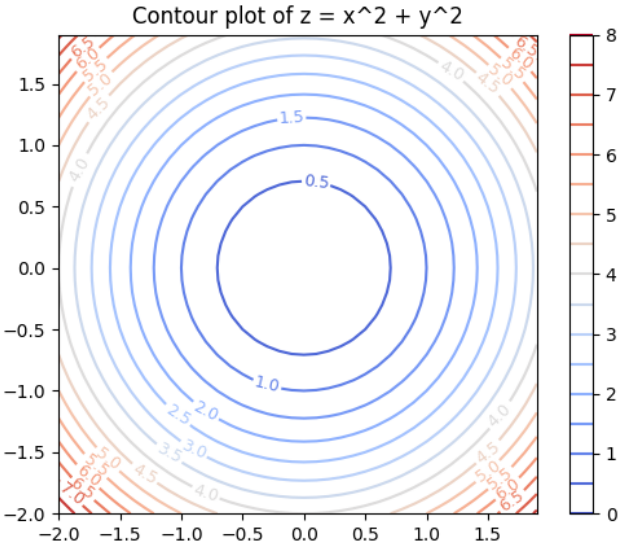
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

levels = np.arange(0, 8.5, 0.5)

fig = plt.figure()
ax = fig.add_subplot(111)

CS = ax.contour(X, Y, Z, levels, cmap=mpl.cm.coolwarm)
ax.set_aspect("equal", adjustable="box")

plt.clabel(CS, inline=True, fontsize=9)
plt.colorbar(CS)
plt.title("Contour plot of z = x^2 + y^2")
plt.show()
```



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

4. Elementary data analysis with Pandas

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- **`pandas`** is a powerful library for working with structured tabular data
  - It allows using datatypes (such as **`DataFrame`**) more powerful than dictionaries
  - It allows quick analysis, filtering or visualizing tabular data
- Let's recall how a **`dict`** works.

```
species_age={'E.coli':1, 'Yeast':2, 'Fruit fly':14, 'Zebrafish':90}; print(species_age)
```

```
{'E.coli': 1, 'Yeast': 2, 'Fruit fly': 14, 'Zebrafish': 90}
```

```
# Accessing the age of zebrafish
age_fish=species_age['Zebrafish']; print(age_fish)
```

```
90
```

```
# Add new entries
species_age['Mouse']=120; species_age['Yeast']=3; print(species_age)
```

```
{'E.coli': 1, 'Yeast': 3, 'Fruit fly': 14, 'Zebrafish': 90, 'Mouse': 120}
```

```
# Delete a key
del species_age['E.coli']; print(species_age)
```

```
{'Yeast': 3, 'Fruit fly': 14, 'Zebrafish': 90, 'Mouse': 120}
```

- Exercise 1: Create a dictionary to record the pH levels of different water samples from two tanks, with values of 6.8 and 6.5. Update one of the values to 6.7 and print the dictionary.

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- Let's convert the **`dict`** into a **`pandas`** object **`DataFrame`**.

```
species_age={'E.coli':1, 'Yeast':2, 'Fruit fly':14, 'Zebrafish':90};
```

```
import pandas as pd

list_dict=list(species_age.items())

df=pd.DataFrame(list_dict, columns=['Species', 'Age (days)'])
```

```
df
```

	Species	Age (days)
0	E.coli	1
1	Yeast	2
2	Fruit fly	14
3	Zebrafish	90

These indices are not part of the actual data

```
print(df)
```

```
     Species  Age (days)
0     E.coli           1
1      Yeast           2
2  Fruit fly          14
3  Zebrafish          90
```

This example shows that **`DataFrame`** allows one to visualize data in tabular form.

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- Another efficient way is to define the columns as lists, convert them to **`dict`**, then into **`DataFrame`**.

```
species_name=['E.coli', 'Yeast', 'Fruit fly', 'Zebrafish']
age=[1,2,14,90]

data={
    'Species':species_name,
    'Age (days)': age
}

df=pd.DataFrame(data)

df
```

	Species	Age (days)
0	E.coli	1
1	Yeast	2
2	Fruit fly	14
3	Zebrafish	90

- One can access non-numerical and numerical columns of a **`DataFrame`** as follows.

```
df['Species']
```

```
0       E.coli
1        Yeast
2    Fruit fly
3    Zebrafish
Name: Species, dtype: object
```

```
import numpy as np

age=df['Age (days)']
print(np.array(age))
```

```
[ 1  2 14 90]
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- Converting multiple columns into a **`DataFrame`**, starting from a **`dict`**

```
data={
    'Species':['E.coli', 'Yeast', 'Zebrafish'],
    'Doubling time (hrs)': [0.5, 1.5, 48],
    'Temperature (C)': [37, 30, 28]
}

df=pd.DataFrame(data); print(df)
```

```
     Species  Doubling time (hrs)  Temperature (C)
0     E.coli                  0.5               37
1      Yeast                  1.5               30
2  Zebrafish                 48.0               28
```

- Converting multiple columns defined as lists into a **`DataFrame`**.

```
# header of each column (or column name)
columns=['Species', 'Doubling time (hrs)', 'Temperature (C)']
# column-1
species=['E.coli', 'Yeast', 'Zebrafish']
# column-2
doubling_time=[0.5, 1.5, 48]
# column-3
temperature=[37, 30, 28]

data=dict(zip(columns, [species, doubling_time, temperature]))

df=pd.DataFrame(data); print(df)
```

```
     Species  Doubling time (hrs)  Temperature (C)
0     E.coli                  0.5               37
1      Yeast                  1.5               30
2  Zebrafish                 48.0               28
```

- Exercise 2: Create a DataFrame to store blood glucose levels of three patients using columns ‘Patient ID’ and ‘Glucose (mg/dL)’.

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- Use the **`iloc`** attribute to extract the value of a row or/and column of a **`DataFrame`**

```
df.iloc[0] # first row in the DataFrame
```

```
Species                E.coli
Doubling time (hrs)       0.5
Temperature (C)            37
Name: 0, dtype: object
```

```
df.iloc[0,0] # first row and first column
```

```
'E.coli'
```

```
df.iloc[0,1] # first row and second column
```

```
0.5
```

- Occasionally, one may want to use string-based indices as follows

```
df.index=['Sample 1', 'Sample 2', 'Sample 3']
df
```

- Use **`loc`** attribute to extract a value of a row (if the row index is a string)

```
df.loc['Sample 1']
```

```
Species                E.coli
Doubling time (hrs)       0.5
Temperature (C)            37
Name: Sample 1, dtype: object
```

```
df.loc['Sample 1'].iloc[1] # using row index (string) and column index (number)
```

```
0.5
```

```
df.loc['Sample 1', 'Doubling time (hrs)'] # Using row index (string) and column name (string)
```

```
0.5
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- One can also use one of the columns as index

```
df.set_index('Species', inplace=True); print(df)
```

```
           Doubling time (hrs)  Temperature (C)
Species
E.coli                     0.5               37
Yeast                      1.5               30
Zebrafish                 48.0               28
```

```
df.loc['Yeast']
```

```
Doubling time (hrs)     1.5
Temperature (C)        30.0
Name: Yeast, dtype: float64
```

- Data with multiple rows and columns, stored in a csv file can be loaded using the **`read_csv`** function

```
df=pd.read_csv('amino_acids.csv')

df
```

	Amino Acid	Molecular Weight (Da)	Full Name
0	A	89.09	Alanine
1	C	121.16	Cysteine
2	D	133.10	Aspartic Acid
3	E	147.13	Glutamic Acid
4	F	165.19	Phenylalanine

- If the csv file does not contain a header row, one should use **`pd.read_csv('file.csv', header=None)`**

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- One can define different rows while reading the csv file

```
df=pd.read_csv('amino_acids.csv', names=['Code', 'Wt', 'Name'], header=0)
df
```

	Code	Wt	Name
0	A	89.09	Alanine
1	C	121.16	Cysteine
2	D	133.10	Aspartic Acid
3	E	147.13	Glutamic Acid
4	F	165.19	Phenylalanine
5	G	75.07	Glycine

- Exercise 3: Define a list containing values of hydrophobicity of amino acids as follows and add it to the DataFrame

  `hydrophobicity=[1.8, 2.5, -3.5, -3.5, 2.8, -0.4, -3.2, 4.5, -3.9, 3.8, 1.9, -3.5, -1.6, -3.5, -4.5, -0.8, -0.7, 4.2, -0.9, -1.3]`

```
hydrophobicity=[1.8, 2.5, -3.5, -3.5, 2.8,
                -0.4, -3.2, 4.5, -3.9, 3.8,
                1.9, -3.5, -1.6, -3.5, -4.5,
                -0.8, -0.7, 4.2, -0.9, -1.3]
df['Hp']=hydrophobicity

df.round(2)
```

	Code	Wt	Name	Hp
0	A	89.09	Alanine	1.8
1	C	121.16	Cysteine	2.5
2	D	133.10	Aspartic Acid	-3.5

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- *Analysis with pandas:* Once a DataFrame is created it is possible to do various analysis of the data. Let's calculate Pearson correlation between numerical columns

```
df.corr(numeric_only=True)
```

	Wt	Hp
Wt	1.000000	-0.270997
Hp	-0.270997	1.000000

```
df.corr(numeric_only=True).round(2)
```

	Wt	Hp
Wt	1.00	-0.27
Hp	-0.27	1.00

- One can quickly calculate the mean, standard deviation and many statistical parameters using the **`describe`** function

```
df.describe()
```

	Wt	Hp
count	20.000000	20.000000
mean	136.901000	-0.490000
std	30.864284	2.986972
min	75.070000	-4.500000
25%	118.627500	-3.500000
50%	132.610000	-0.850000
75%	150.697500	2.050000
max	204.230000	4.500000

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- *Plotting with pandas:* Scatter matrix plot is a quick way to visualize the distribution of values in each column as well as the correlation between different columns with numerical data

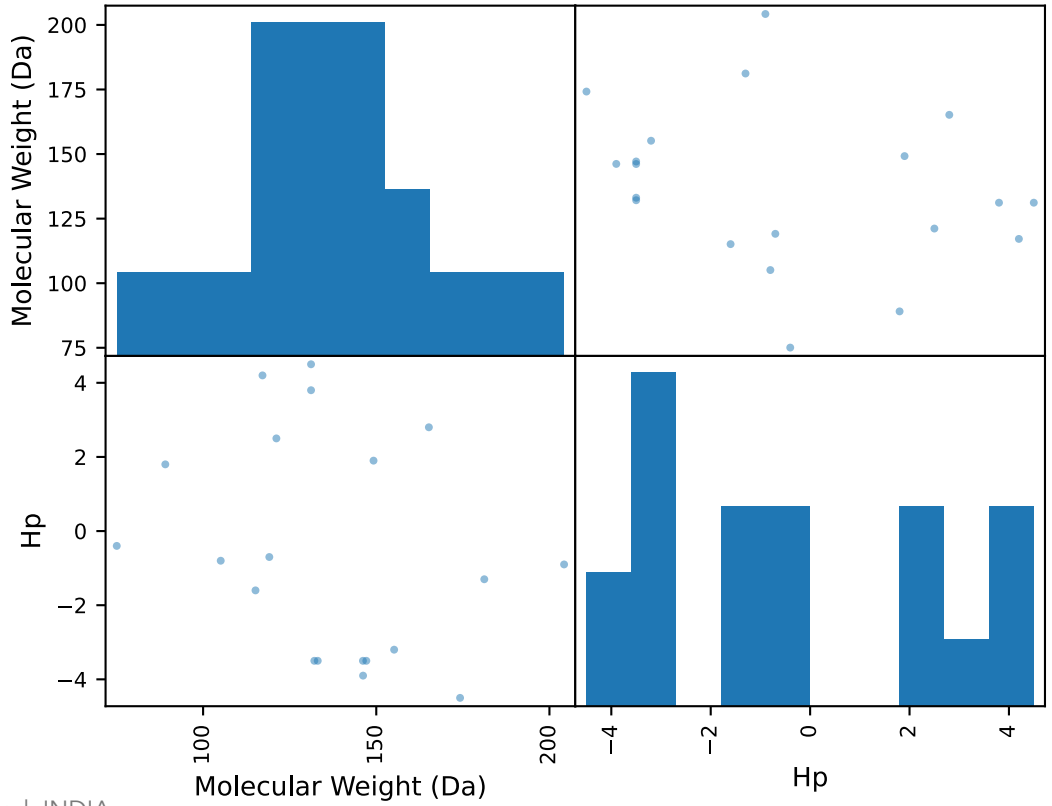
```
import pandas as pd
import matplotlib.pyplot as plt
from pandas.plotting import scatter_matrix

df=pd.read_csv('amino_acids.csv')

hydrophobicity=[1.8, 2.5, -3.5, -3.5, 2.8, -0.4, -3.2, 4.5, -3.9, 3.8,
                1.9, -3.5, -1.6, -3.5, -4.5, -0.8, -0.7, 4.2, -0.9, -1.3]
df['Hp']=hydrophobicity

scatter_matrix(df)

plt.savefig('scatter.pdf')
```



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Counting roses: A mini-project with **pandas**

- *Rosa.csv:* The dataset Rosa contains the scientific names of all species of the rose plant in a csv format. The dataset was collected on 12 Jan 2024 from the Wikipedia article: https://en.wikipedia.org/wiki/List_of_Rosa_species#cite_note-30002432-2-1. Let's read (or load) this file in Python and do some statistics.
- The csv file contains several rows (first row is the header **genus,hyb,species**). The second column is empty if the species is pure and it contains **x** if the species is hybrid.

```
import pandas as pd

df = pd.read_csv('./Rosa.csv')
```

- **print (df)** displays the DataFrame
- **df.shape** gives the number of rows and columns as a tuple. **df.shape[0]** gives the number of rows and **df.shape[1]** gives the number of columns.
- **df.info** gives a summary of the number and names of rows and columns
- **df.describe** gives a statistical summary
- **df.head(5)** lists the top-5 entries; **df.tail(5)** lists the bottom-5 entries
- **df_hybrid=df[df['hyb'] == 'x']** contains a DataFrame of hybrid species
- **df_pure=df[df['hyb'] != 'x']** contains a DataFrame of pure species
- **df_pure=df[df[‘hyb’].isna()]** contains a DataFrame where the entries of the column **hyb** are not available (i.e. pure species)
- **df.dropna()** returns a DataFrame all rows with missing entries in any column is removed. To make a copy of this, use **df_clean=df.dropna().copy()**
- **df.isna()** shows which columns have values missing. **df.isna().sum()** gives a shorter summary
- Replace empty entries with a value **df.fillna("None")**

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- Plot a histogram of the number of pure and hybrid species as follows.

```
import matplotlib.pyplot as plt

count_pure = df_pure.shape[0]
count_hyb = df_hybrid.shape[0]
counts = [count_hyb, count_pure]

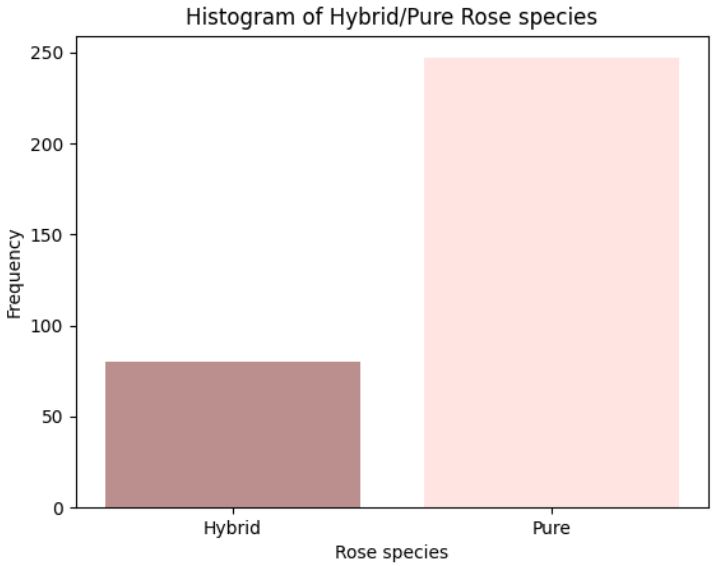
labels = ['Hybrid', 'Pure']

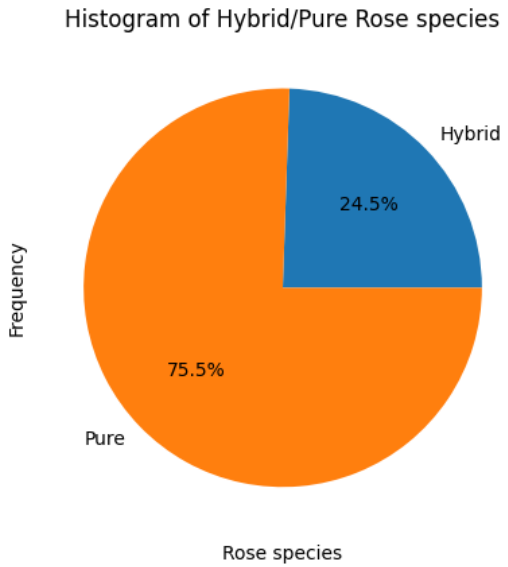
mycolors = ['rosybrown', 'mistyrose'] # https://matplotlib.org/2.0.2/examples/color/named_colors.html

plt.bar(labels, counts, color=mycolors)
plt.xlabel('Rose species')
plt.ylabel('Frequency')
plt.title('Histogram of Hybrid/Pure Rose species')

plt.savefig('histogram_rose.png')

plt.show()
```





- Plot a pie chart by replacing **`plt.bar(labels, counts, color=mycolors)`** with **`plt.pie(counts, labels=labels, autopct='%1.1f%%')`**

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Import datasets from **sklearn**

- Get the list of all datasets available in scikit-learn module as follows

```
import sklearn as sk

dataset_functions = [
    name for name in dir(sk.datasets)
    # List all functions that start with "load_" or "fetch_"
    if name.startswith(("load_", "fetch_"))
]

print(dataset_functions)
```

- Load **iris** and **california_housing** datasets from sklearn

```
data_iris = sk.datasets.load_iris()
df = pd.DataFrame(data_iris.data, columns=data_iris.feature_names)
df.head(5)

data_housing = sk.datasets.fetch_california_housing()
df = pd.DataFrame(data_housing.data, columns=data_housing.feature_names)
df.head(5)
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Import datasets from **seaborn**

- Get the list of all datasets available in Seaborn module as follows

```
import seaborn as sns
sns.get_dataset_names()
```

- Inspect the **iris** dataset imported from seaborn

```
df = sns.load_dataset("iris")
df.head(5)
```

- Inspect the **penguin** dataset imported from seaborn

```
df = sns.load_dataset("penguin")
df.head(5)
```



Iris Versicolor **Iris Setosa** **Iris Virginica**



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Practicals 2: Penguin dataset

1. Repeat the basic analysis done for the Rosa dataset for **`penguin`** dataset imported from seaborn
2. Remove rows with missing values and make a clean copy
   `df_clean = df.dropna().copy()`
3. Calculate how many species are there in the dataset per island
   `df_clean["island"].value_counts()`
4. Calculate mean body mass of penguns per species
   `df_clean.groupby("species")["body_mass_g"].mean()`
5. Add another column for bill ratio

```
df_clean["bill_ratio"]=(df_clean["bill_length_mm"]/df_clean["bill_depth_mm"] )
df_clean.head(5)
```

6. In the above step, bill ratio will be listed as the last column. If you want to list it next to bill depth, do the following

```
df_clean = df.dropna().copy()

bill_ratio = (
    df_clean["bill_length_mm"] /
    df_clean["bill_depth_mm"]
)

df_clean.insert(4, "bill_ratio", bill_ratio)

df_clean.head(5)
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

5. Mathematical modeling

(Differential equations, Curve-fitting:
Growth models, population, reaction rate, allometry)

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

■ Growth/Decay functions are solutions of differential equations

- Consider the exponential growth function $P(t) = P_0 e^{At}$. By substituting $t = 0$, we see that at the initial time, the population is P_0. If the growth rate is positive, $A > 0$, the population will increase exponentially from the initial value.
- Can the same information be presented in a different form? Yes, in a differential form as follows $\frac{dP(t)}{dt} = AP(t)$. This equation (the growth equation in the differential form, or the growth differential equation) states that the rate of increase in the population is proportional to the population at that time.
- In other words, $P(t) = P_0 e^{At}$ is the solution of the differential equation $\frac{dP(t)}{dt} = AP(t)$.

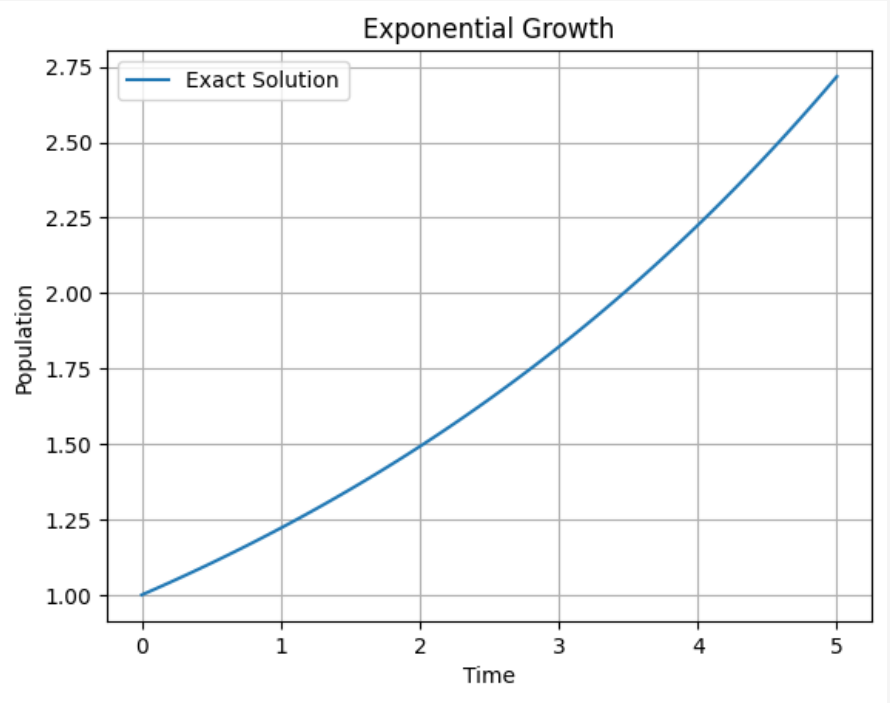
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# Analytic solution
def solution(t, P0, A):
    return P0 * np.exp(A * t)

t = np.linspace(0, 5, 100)   # Time points
A = 0.2                      # Growth rate
P0 = 1                       # Initial population

Pt_exact = solution(t, P0, A) # Exact solution

plt.plot(t, Pt_exact, label='Exact Solution')
plt.xlabel('Time')
plt.ylabel('Population')
plt.title('Exponential Growth')
plt.legend()
plt.grid(True)
plt.savefig("pop_exact.png", bbox_inches='tight')
plt.show()
```



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

- **Find the solution numerically by solving the ODE using `scipy.integrate.odeint`**
  - Recall that the exact solution of $\frac{dP(t)}{dt} = AP(t)$ is $P(t) = P_0 e^{At}$. Since the solution is a function of the initial population, P_0, we say that the ODE has infinite solutions in general, and a particular solution for a given value of P_0.
  - We will solve the ODE numerically as an initial-value problem, for which we need the initial value of the solution, i.e., $P(0)$.
  - To solve $\frac{dy}{dx} = f(x, y)$, all we need is to code the R.H.S. $f(x, y)$.

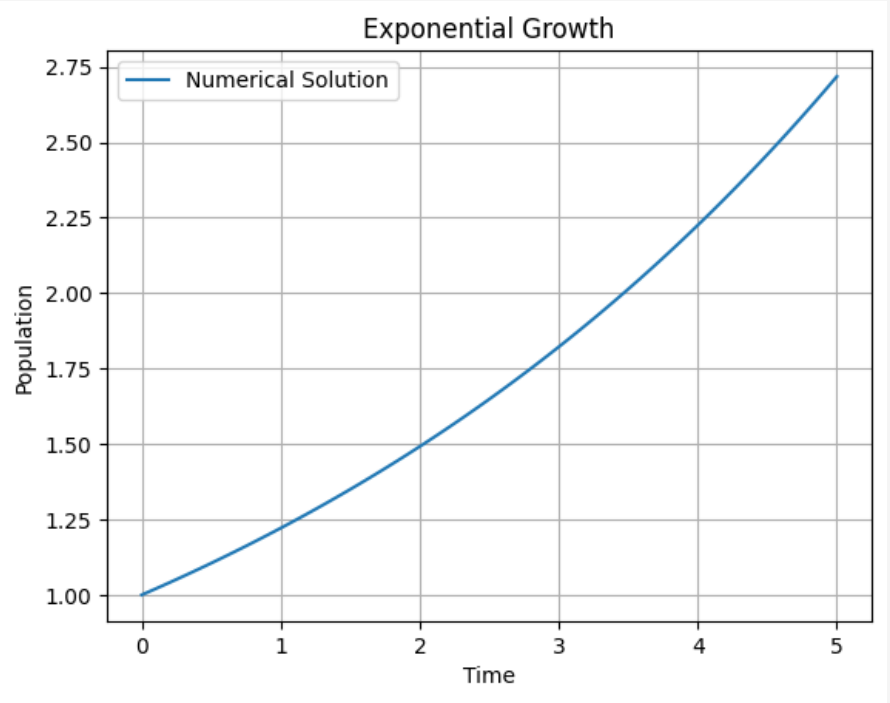
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# Growth model, R.H.S
def growth(P, t, A):
    return A * P

t = np.linspace(0, 5, 100)   # Time points
A = 0.2                      # Growth rate
P0 = 1                       # Initial population

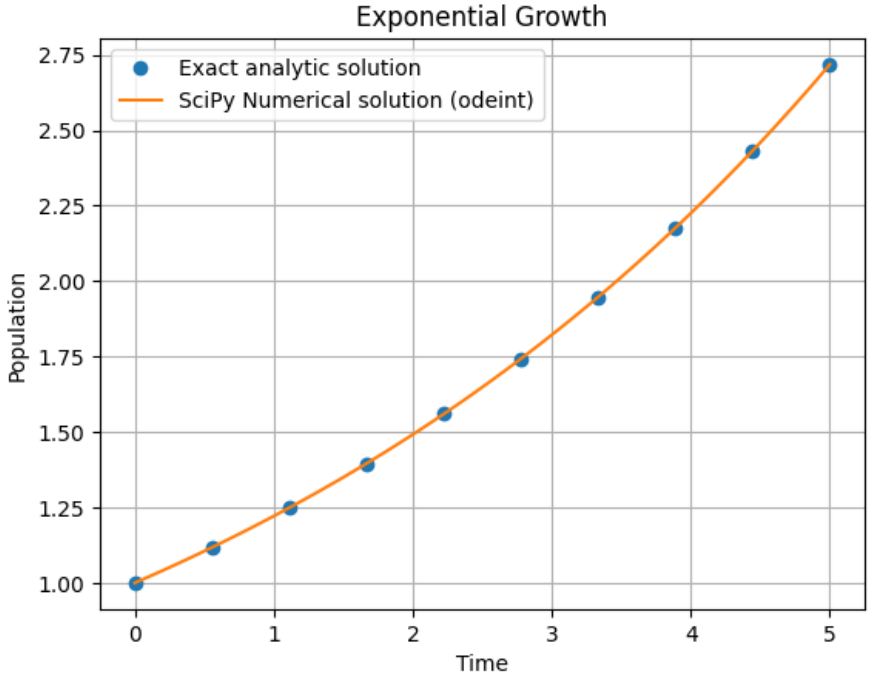
Pt_numerical = odeint(growth, P0, t, args=(A,)) # Numerical solut

plt.plot(t, Pt_numerical, label='Numerical Solution')
plt.xlabel('Time')
plt.ylabel('Population')
plt.title('Exponential Growth')
plt.legend()
plt.grid(True)
plt.savefig("pop_num.png", bbox_inches='tight')
plt.show()
```



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

■ Plot both exact and numerical solution together




```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# Analytic solution
def solution(t, P0, A):
    return P0 * np.exp(A * t)

# Exponential growth model
def growth(P, t, A):
    return A * P


A = 0.2                      # Growth rate
P0 = 1                       # Initial population

t_discrete = np.linspace(0, 5, 10) # Fewer time values to plot exact solution
Pt_exact = solution(t_discrete, P0, A) # Exact analytic solution

t = np.linspace(0, 5, 100)   # Time points
Pt_numerical = odeint(growth, P0, t, args=(A,)) # Solve the ODE numerically

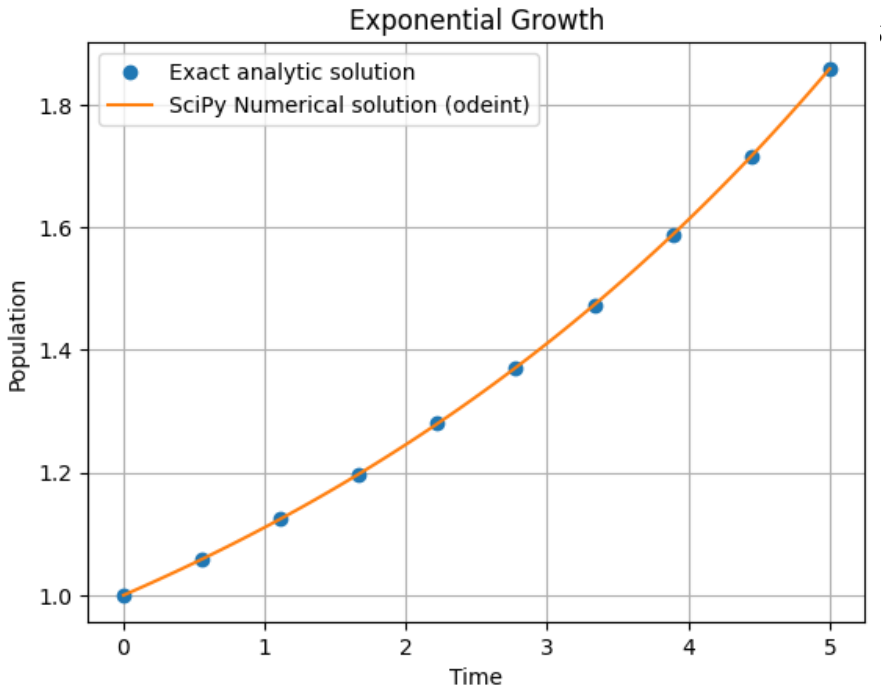
plt.plot(t_discrete, Pt_exact, label='Exact analytic solution', linestyle='', marker='o')
plt.plot(t, Pt_numerical, label='SciPy Numerical solution (odeint)')

plt.xlabel('Time')
plt.ylabel('Population')
plt.title('Exponential Growth')
plt.legend()
plt.grid(True)
plt.savefig("pop_both.png", bbox_inches='tight')
plt.show()
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Growth with an attrition

- Suppose the population of a species grows exponentially but a certain number of members of the population die at a constant death rate (*e.g.* 3 members in a minute). Then, we need to include a parameter that accounts for the constant decline in the population. The differential equation for this growth process is $\frac{dP(t)}{dt} = AP(t) - B$, where B is the a that the population decreases.
- The exact solution is $P(t) = (P_0 - B/A)e^{At} + B/A$.




```
# Exponential growth with constant attrition
def growth(P, t, A, B):
    return A * P - B

# Analytic solution
def solution(t, P0, A, B):
    return (P0 - B/A) * np.exp(A * t) + B/A

# A: Growth rate, B: Attrition rate, P0: Initial population
A, B, P0 = 0.2, 0.1, 1

t_discrete = np.linspace(0, 5, 10) # Fewer time values to plot ex
Pt_exact = solution(t_discrete, P0, A, B) # Exact analytic soluti

t = np.linspace(0, 5, 100)   # Time points
Pt_numerical = odeint(growth, P0, t, args=(A,B)) # Solve the ODE numerically

plt.plot(t_discrete, Pt_exact, label='Exact analytic solution', linestyle='', marker='o')
plt.plot(t, Pt_numerical, label='SciPy Numerical solution (odeint)')

plt.xlabel('Time')
plt.ylabel('Population')
plt.title('Exponential Growth')
plt.legend()
plt.grid(True)
plt.savefig("pop_with_attrition_both.png", bbox_inches='tight')
plt.show()
```

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Compare growth without attrition ($B = 0$) and with attrition ($B \neq 0$)

```
# Exponential growth with constant attrition
def growth(P, t, A, B):
    return A * P - B

# Analytic solution
def solution(t, P0, A, B):
    return (P0 - B/A) * np.exp(A * t) + B/A

# A: Growth rate, B: Attrition rate, P0: Initial population
A, B, P0 = 0.2, 0.1, 1

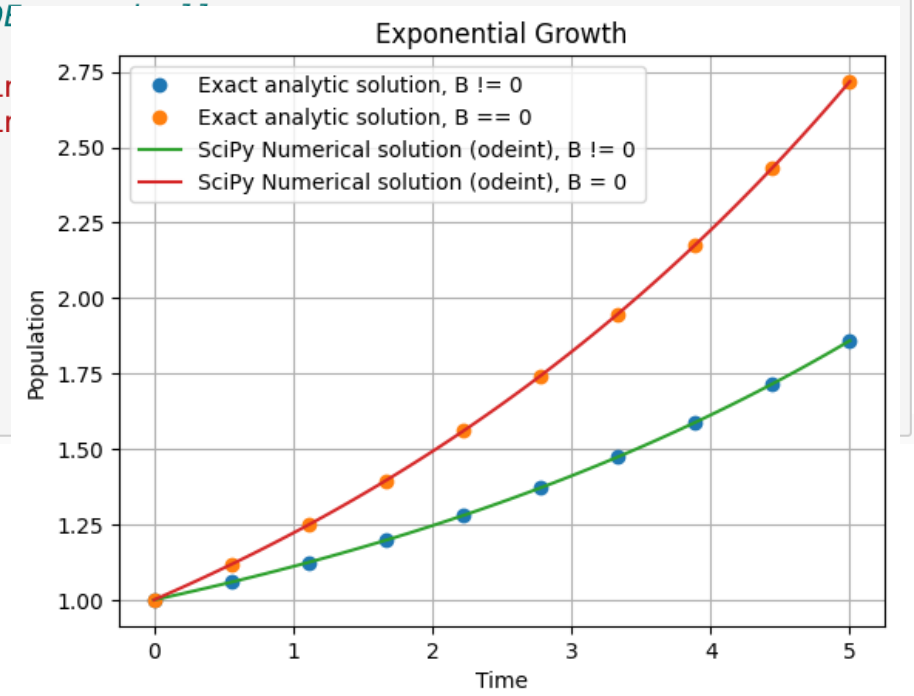
t_discrete = np.linspace(0, 5, 10) # Fewer time values to plot exact solution
Pt_exact_1 = solution(t_discrete, P0, A, B) # Exact analytic solution, B /= 0
Pt_exact_2 = solution(t_discrete, P0, A, 0) # Exact analytic solution, B = 0

plt.plot(t_discrete, Pt_exact_1, label='Exact analytic solution, B != 0', linestyle='', marker='o')
plt.plot(t_discrete, Pt_exact_2, label='Exact analytic solution, B == 0', linestyle='', marker='o')

t = np.linspace(0, 5, 100)   # Time points
Pt_numerical1 = odeint(growth, P0, t, args=(A,B)) # Solve the ODE numerically
Pt_numerical2 = odeint(growth, P0, t, args=(A,0)) # Solve the ODE

plt.plot(t, Pt_numerical1, label='SciPy Numerical solution (odeint
plt.plot(t, Pt_numerical2, label='SciPy Numerical solution (odeint

plt.xlabel('Time')
plt.ylabel('Population')
plt.title('Exponential Growth')
plt.legend()
plt.grid(True)
plt.savefig("pop_with_attrition_both.png", bbox_inches='tight')
plt.show()
```

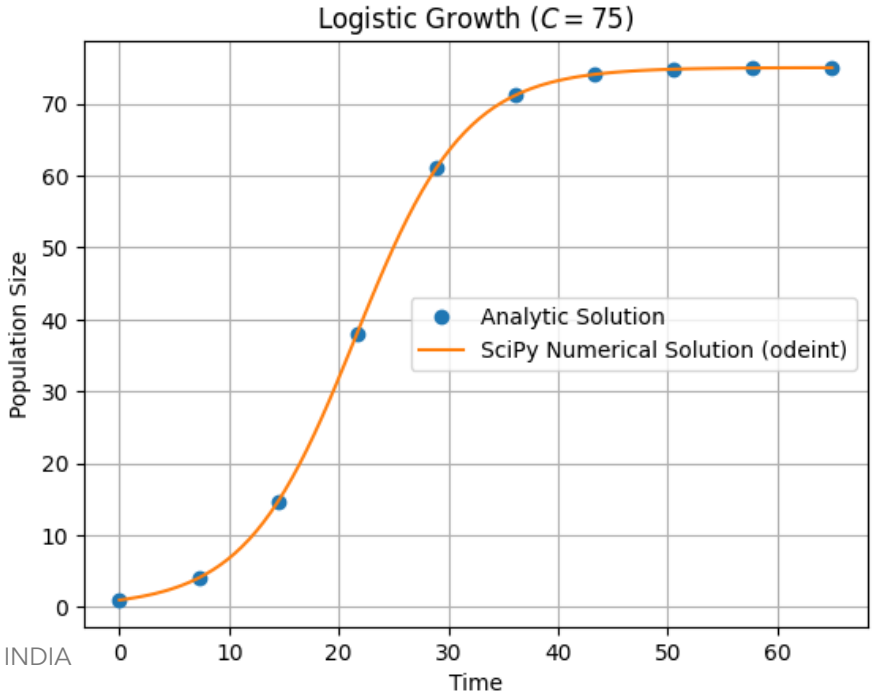


© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Practicals 3: Logistic growth

- Suppose the population of a species grows exponentially at the beginning but the resources in the environment is limited, then the corresponding growth differential equation is $\frac{dP(t)}{dt} = AP(t)\left[1 - \frac{P(t)}{C}\right]$, where C is the carrying capacity of the environment (maximum population size that the environment can support). The exact solution is $P(t) = \frac{C}{1 + \frac{C - P_0}{P_0} e^{-At}}$. For large values of t, e^{-At} approaches zero, hence the population will approach C.

1. Modify the code given for the previous case and plot the exact solution of the logistic differential equation for $P_0 = 1$ (initial population), $A = 0.2$ (growth rate), and $C = 75$ (carrying capacity).
2. Using the same constants, solve the logistic differential equation numerically using the function **`scipy.integrate.odeint`** and plot the numerical solution alongside the exact, analytic solution as shown below.



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Reaction models

One variable ODE: One-step reaction

- Let's solve $A \xrightarrow{k} B$ with the initial concentration, $A[0] = 1$, and rate constant, $k = 1$. We know that $A[t] + B[t] = 1$, so $B[t] = 1 - A[t]$.
- To find $A[t]$, we need to solve $\frac{dA(t)}{dt} = -k_1 A(t)$.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

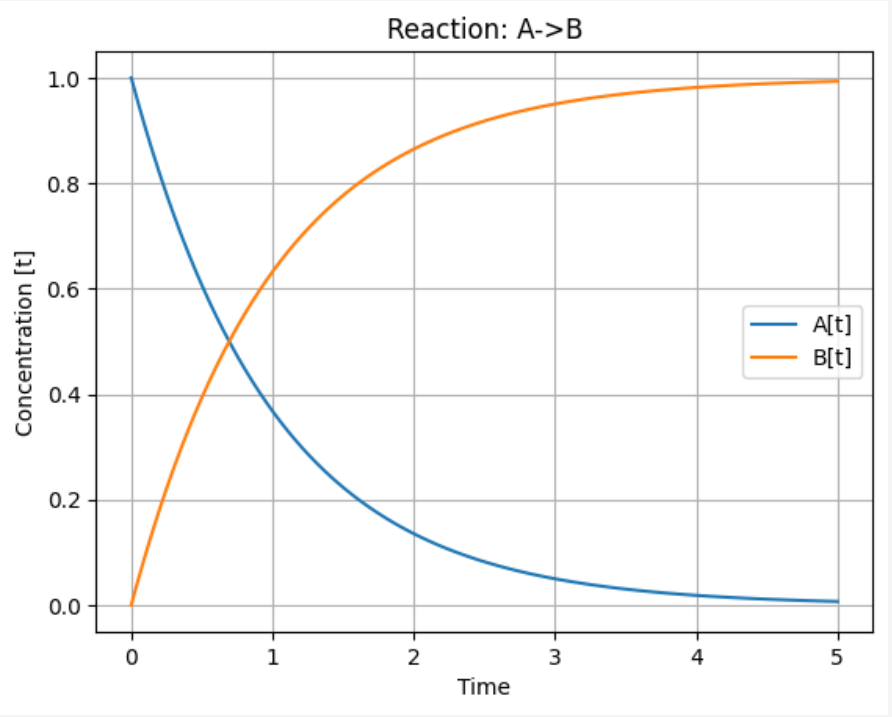
def rxn_AB(A, t, k1):
    dAdt = -k1 * A   # this is the RHS of d[A]/dt=-k1[A]
    return dAdt

A0, k1 = 1, 1

t = np.linspace(0, 5, 100)
A_t = odeint(rxn_AB, A0, t, args=(k1,))
B_t = A0-A_t

plt.plot(t, A_t, label='A[t]')
plt.plot(t, B_t, label='B[t]')

plt.xlabel('Time')
plt.ylabel('Concentration [t]')
plt.title('Reaction: A->B')
plt.legend()
plt.grid(True)
plt.savefig("rxn_AB.png", bbox_inches='tight')
plt.show()
```



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Two variables ODE: Reversible reaction

- Let's solve $A \underset{k_b}{\overset{k_f}{\rightleftharpoons}} B$ with the initial concentration, $A[0] = 1$, and rate constant, $k_f = k_b = 1$. Let's solve this without assuming $A[t] + B[t] = 1$
- To find $A[t]$ and $B[t]$, we need to solve the coupled ODEs

$$\begin{aligned} \frac{dA(t)}{dt} &= -k_f A(t) + k_b B(t) \\ \frac{dB(t)}{dt} &= -k_b B(t) + k_f A(t) \end{aligned}.$$

- To solve the two-variable ODE, we can write the coupled ODE as $\frac{d\mathbf{P}(t)}{dt} = \mathbf{f}(\mathbf{P}(t))$, or

$$\begin{aligned} \frac{dP_1(t)}{dt} &= f_1(P_1(t), P_2(t)) \\ \frac{dP_2(t)}{dt} &= f_2(P_1(t), P_2(t)) \end{aligned}.$$

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

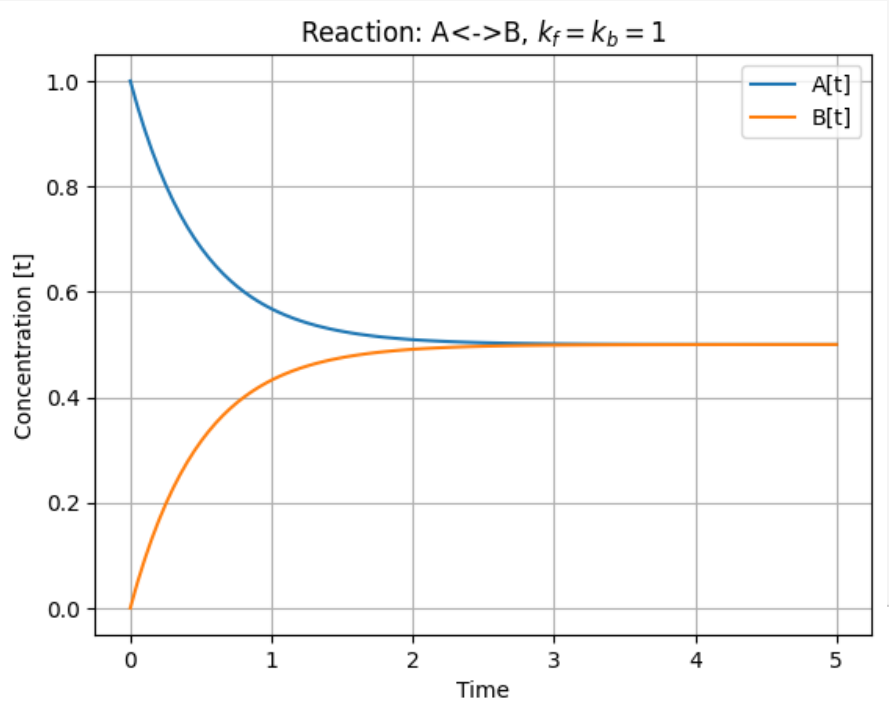
def rxn_AB_rev(P, t, kf, kb):
    A, B = P[0], P[1]
    dAdt = -kf * A + kb * B   # this is the RHS of d[A]/dt=-kf[A] + kb[B]
    dBdt = -kb * B + kf * A   # this is the RHS of d[B]/dt=-kb[B] + kf[A]
    return dAdt, dBdt

A0, B0, kf, kb = 1, 0, 1, 1

t = np.linspace(0, 5, 100)
P_t = odeint(rxn_AB_rev, [A0, B0], t, args=(kf, kb))

A_t, B_t = P_t[:,0], P_t[:,1]
plt.plot(t, A_t, label='A[t]')
plt.plot(t, B_t, label='B[t]')

plt.xlabel('Time')
plt.ylabel('Concentration [t]')
plt.title('Reaction: A<->B, $k_f=k_b=1$')
plt.legend()
plt.grid(True)
plt.savefig("rxn_AB_rev.png", bbox_inches='tight')
plt.show()
```



© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Predator-prey dynamics

Lotka-Volterra equations

- The Lotka-Volterra model for predator-prey dynamics is a set of differential equations that describe the interactions between two species in an ecosystem, typically a prey species (denoted as R) and a predator species (denoted as P).
- The equations governing this process are as follows:

$$\frac{dR(t)}{dt} = rR(t) - aR(t)P(t) \quad \text{and} \quad \frac{dP(t)}{dt} = -sP(t) + bR(t)P(t)$$

- r is the reproduction rate of prey
  a is the rate of consumption of prey by predators (interaction)
  s is the death rate of predator
  b is the reproduction rate of predator (as an outcome of consumption of prey, interaction)

Exercise

- A forest has 100 rabbits and 5 foxes. The rabbit population increases by 1 per week, and the foxes eat 1 rabbit per 10 weeks. 1 fox dies in 10 weeks due to natural reasons. Assuming that the rabbit is the only source of food for foxes, the fox population increases by 1 per 50 weeks (roughly, a year). Use the following data and solve the Lotka-Volterra equations, and plot the rabbit and fox populations for 100 weeks (roughly 2 years).
  - $R(0) = 100$, $P(0) = 5$, $r = 1$, $a = 1/10 = 0.1$, $s = 1/10 = 0.1$, and $b = 1/50 = 0.02$

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

```python
def lotka_volterra(y, t, r, a, s, b):
    R, P = y
    dRdt = r * R - a * R * P
    dPdt = -s * P + b * R * P
    return [dRdt, dPdt]

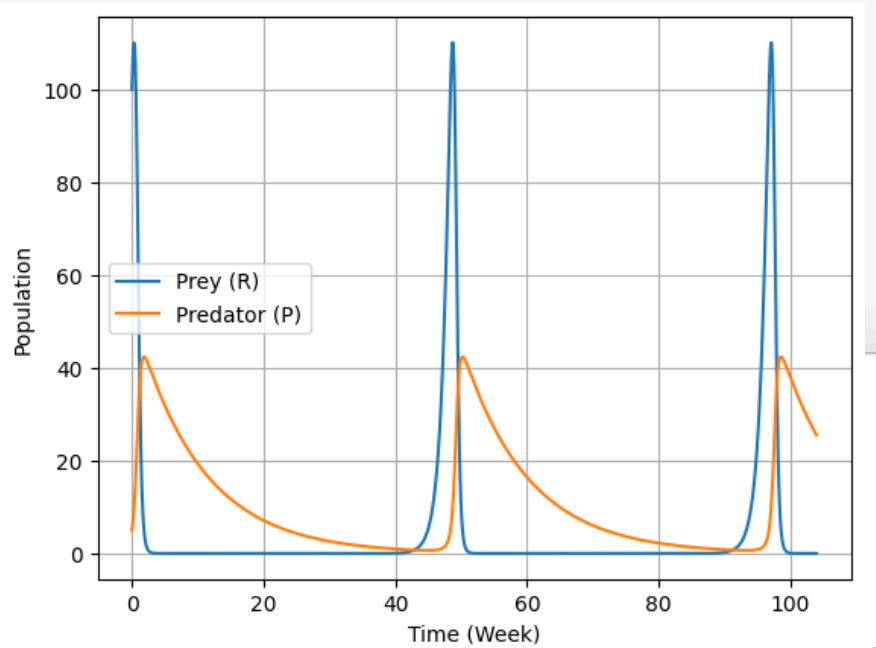
# Initial populations of prey (R) and predator (P)
R0 = 100
P0 = 5

# Rate constants
r = 1.0
a = 0.1
s = 0.1
b = 0.02

# Time points
t = np.linspace(0, 104, 1000)

# Solve the Lotka-Volterra equations
solution = odeint(lotka_volterra, [R0, P0], t, args=(r, a, s, b))

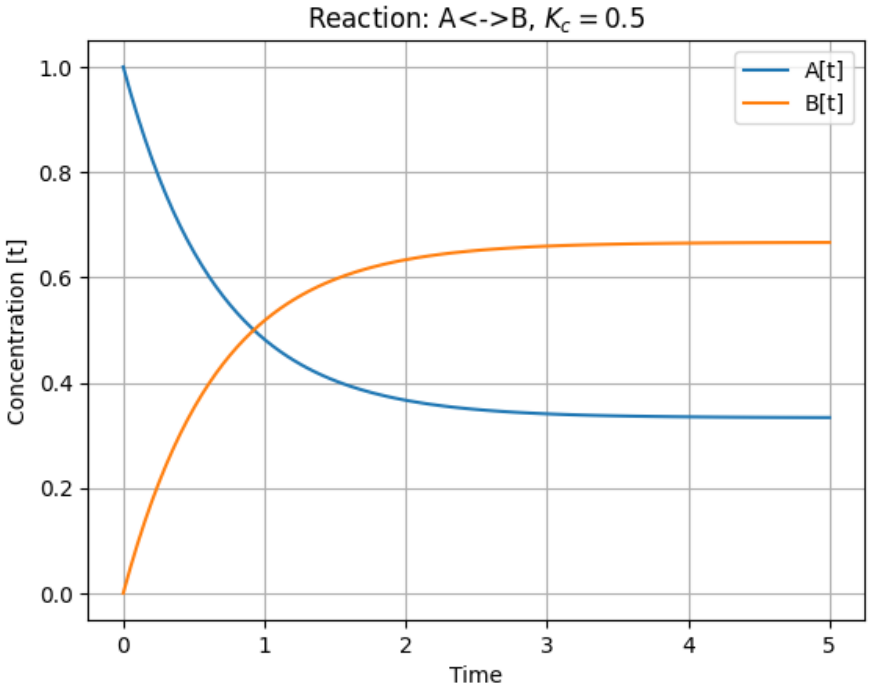
# Plot the results
plt.plot(t, solution[:, 0], label='Prey (R)')
plt.plot(t, solution[:, 1], label='Predator (P)')
plt.xlabel('Time (Week)')
plt.ylabel('Population')
plt.legend()
plt.grid(True)
plt.savefig('PredatorPrey.png',bbox_inches='tight')
plt.show()
```

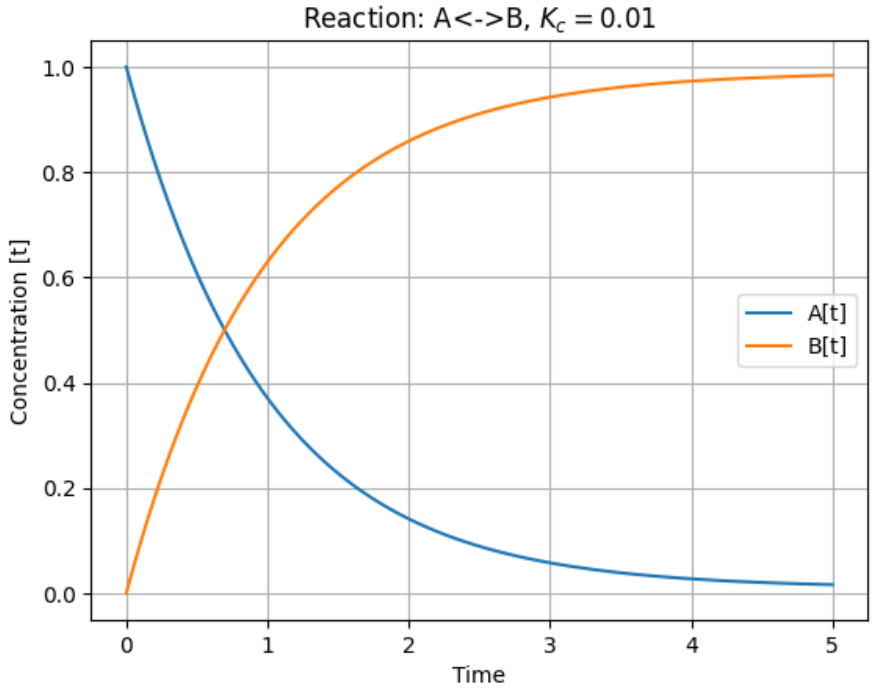


© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Practicals 4: Equilibrium constant

- Solve the reaction model for the reaction, $A \underset{k_b}{\overset{k_f}{\rightleftharpoons}} B$ with the initial concentration, $A[0] = 1$, and rate constant, and equilibrium constant $K_c = 0.5$. Recall that the definition of equiibrium constant is $K_c = \frac{k_f}{k_b}$. Hint: use $k_f = 1$. At equilibrium (large t), do you expect $A(t) > B(t)$, $A(t) < B(t)$, or $A(t) = B(t)$?
- Repeat the previous problem with a very large equilibrium constant, $K_c = 100$. What is the expected result?

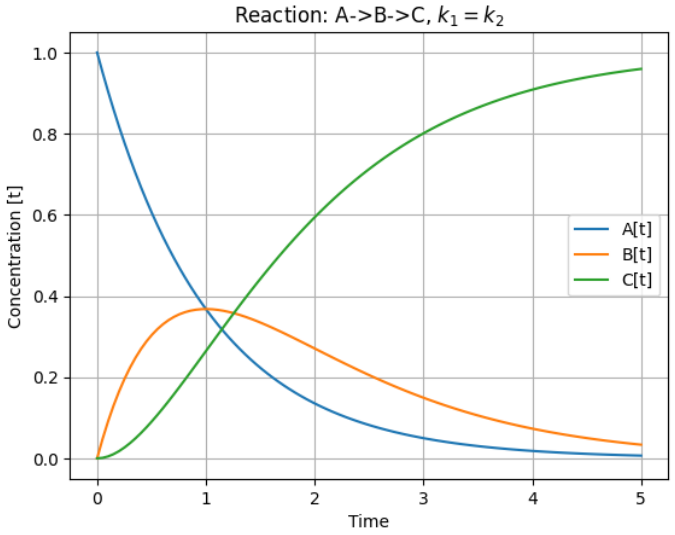


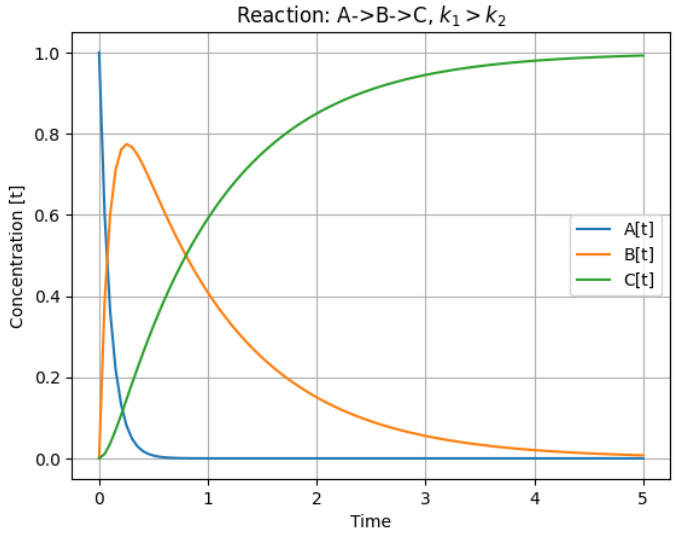


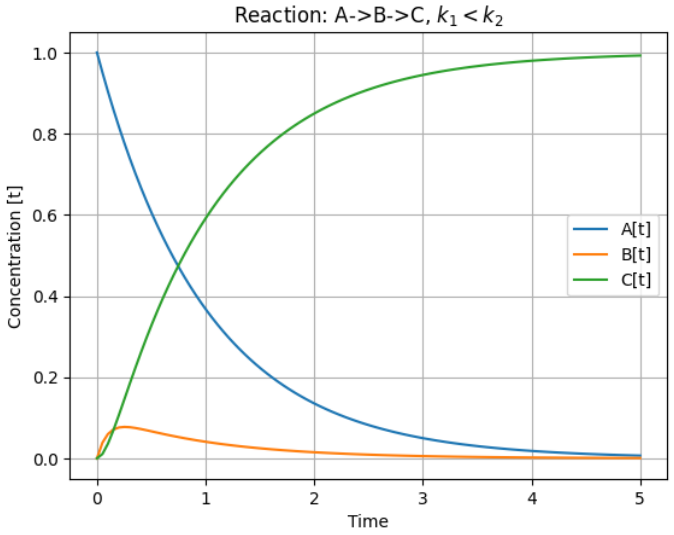
© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Practicals 5: Two step $A \rightarrow B \rightarrow C$ reaction

- Find $A(t)$, $B(t)$, and $C(t)$ for the reaction $A \rightarrow B \rightarrow C$, with $A[0] = 1$ and $A(t) + B(t) + C(t) = A[0]$.
  - Solve the ODE with $k_1 = k_2 = 1$
  - Solve the ODE with $k_1 = 1$ and $k_2 = 10$
  - Solve the ODE with $k_1 = 10$ and $k_2 = 1$
- Which of the following cases corresponds to steady-state approximation when the two-step reaction resembles a one-step reaction.







© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA

Assignment

- **Simple infection model for interacting population**

  Suppose in a gated community, we divide the population into 2 classes: those infected by a virus (I) and those not infected (N) by the virus. The total population is a constant, $T = I(t) + N(t)$. Rate of increase in the number of infections is given by $\frac{dI(t)}{dt} = \alpha I(t)N(t) - \beta I(t)$, where α is the rate of contact between healthy and infected people, and β is the recovery rate of infected people. Assume that at initial time, there were 10 infected individuals, and solve this equation and plot $I(t)$ and $N(t)$ for 4 weeks using $T = 100$, $\alpha = 0.05$ (i.e. 5 infections in 100 contacts in a week), and $\beta = 4.0$ (i.e. 4 recover in a week's time after infection).

- **A hypothetical model for students learning Python**

  Suppose in a class, there are $T = 30$ students, and let $N(t)$ denote the number of students learning python at a given time. Rate in which students learning Python is given by $\frac{dN(t)}{dt} = \alpha N(t)(T - N(t))$, where α is the rate of learning (due to contact between students who have learned Python, $N(t)$, and the students who are yet to learn Python). Using $\alpha = 0.01$ number-per-week, and $N_0 = 2$ (number of students who knew Python at the initial time), plot $N(t)$ and find out when more than half-of-the class would have learned Python.

- **Tumor growth**

  The growth of a tumor (increase in cell count) follows the differential equation $\frac{dP(t)}{dt} = AP(t)\log\left(\frac{C}{P(t)}\right)$, where C is the carrying capacity of the environment (maximum population size that the environment can support). For an initial population, $P_0 = 10$, growth rate $A = 1$, and the carrying capacity $C = 75$, calculate and plot the solution of this differential equation. Does the solution grow faster or slower than the logistic growth with the same parameters?

© 2026, Raghunathan Ramakrishnan, TIFR Hyderabad, INDIA